

Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Mohit Tanwar

Roll No: 2501730174

Email: 2501730174@krmu.edu.in

Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def compare_search_algorithms(arr, target):
    linear_index = -1
    linear_comparisons = 0

    for i in range(len(arr)):
        linear_comparisons += 1
        if arr[i] == target:
            linear_index = i
            break

    binary_index = -1
    binary_comparisons = 0
    low = 0
    high = len(arr)-1

    while low <= high:
        mid = (low + high) // 2
        binary_comparisons += 1

        if arr[mid] == target:
            binary_index = mid
            high = mid - 1
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    if linear_comparisons < binary_comparisons:
        better = "Linear Search"
    elif binary_comparisons < linear_comparisons:
        better = "Binary Search"
    else:
        better = "Both Equal"

    return[
        "Search Comparison Report",
        "Linear Search",
        f"Index: {linear_index}",
        f"Comparisons: {linear_comparisons}",
        "Binary Search",
        f"Index: {binary_index}",
        f"Comparisons: {binary_comparisons}",
        f"Better Algorithm: {better}"
    ]
```

Q2. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```
def compare_bubble_insertion(random_data, sorted_data, reverse_data):
    def bubble_sort(arr):
        a = arr[:]
        comparisons = 0
        swaps = 0
        n = len(a)

        for i in range(n-1):
            swapped = False

            for j in range(n-1-i):
                comparisons += 1

                if a[j] > a[j+1]:
                    a[j], a[j+1] = a[j+1], a[j]
                    swaps += 1
```

```

        swaps += 1
        swapped = True

    if not swapped:
        break

    return a, comparisons, swaps

def insertion_sort(arr):
    a = arr[:]
    comparisons = 0
    shifts = 0

    for i in range(1, len(a)):
        key = a[i]
        j = i - 1

        while j >= 0:
            comparisons += 1

            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1

            else:
                break

        a[j + 1] = key

    return a, comparisons, shifts

datasets = [
    ("Random Dataset", random_data),
    ("Sorted Dataset", sorted_data),
    ("Reverse Dataset", reverse_data)
]

output = ["Sorting Performance Report"]

for name, data in datasets:
    b_sorted, b_comp, b_swaps = bubble_sort(data)
    i_sorted, i_comp, i_shifts = insertion_sort(data)

    output.append(name)
    output.append("Bubble Sorted: " + " ".join(map(str, b_sorted)))
    output.append("Bubble Comparisons: " + str(b_comp))
    output.append("Bubble Swaps: " + str(b_swaps))
    output.append("Insertion Sorted: " + " ".join(map(str, i_sorted)))
    output.append("Insertion Comparisons: " + str(i_comp))
    output.append("Insertion Shifts: " + str(i_shifts))

    if b_comp < i_comp:
        better = "Bubble Sort"
    elif i_comp < b_comp:
        better = "Insertion Sort"
    else:
        better = "Both Equal"

    output.append("Better Algorithm: " + better)

return output

```